

Blue Machines Engineering Assignment

Real-Time Acoustic Intelligence for LiveKit Voice Agents

Context

Build on top of the existing LiveKit backchanneling system.

The current system can listen to a user, decide when a short acknowledgement such as “mm-hmm” may be appropriate, and play it without blocking the normal response.

The next problem is:

Can the agent understand **how the user is speaking**, in real time, and use that signal without increasing conversational latency or making the system fragile?

A transcript captures the words. Audio contains additional signals such as:

- frustration,
- hesitation,
- confidence,
- excitement,
- calmness,
- speaking energy,
- changes in delivery over time.

Build a **streaming acoustic-expression system** and integrate it into the existing LiveKit agent.

The primary signal should come from **audio/prosody rather than transcript text**.

We care as much about your system design and measurement methodology as the model itself.

1. Streaming Acoustic Understanding

Continuously process incoming user audio and produce expression signals **while the user is speaking**.

For example:

Time	Frustration	Uncertainty	Energy
2.0s	0.12	0.31	0.42
2.5s	0.18	0.39	0.48
3.0s	0.41	0.52	0.55
3.5s	0.68	0.43	0.66
4.0s	0.81	0.31	0.72

You decide:

- which signals to expose,
- which model to use,
- window size,
- inference stride,
- smoothing,
- minimum audio required,
- confidence representation.

You may use an existing acoustic model. Training a new model is not required.

Do not describe these outputs as a person's true emotional state. Treat them as **acoustically expressed conversational signals**.

2. Streaming, Not Post-Processing

Do not wait for the user turn to finish.

Your system should produce useful predictions during speech.

Think about the trade-off:

more audio
↓
more stable prediction

versus:

less audio
↓
lower latency

For example:

Window size: 1.5 sec

Stride: 250 ms

Inference: 35 ms

Smoothing: ...

These are examples only. Choose your own design and explain it.

3. Measure True Latency

Instrument the system.

At minimum measure:

Model inference latency

model input ready



prediction produced

Report:

P50

P95

Effective acoustic detection latency

Include the amount of audio required before inference.

If a model needs two seconds of audio, reporting only:

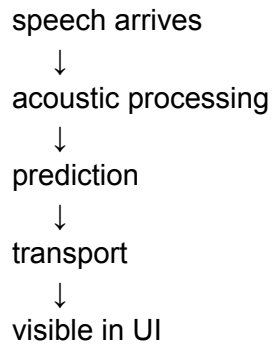
Inference = 30 ms

is misleading.

Explain the **true time from relevant speech occurring to a usable signal becoming available**.

Audio → UI latency

Measure:



Report P50/P95.

4. Acoustic vs Text Baseline

Implement a simple text sentiment baseline.

The acoustic system should work independently of it.

Create scenarios where the transcript is the same but delivery changes.

For example:

“Yeah, that's great.”

with:

genuinely positive

frustrated

flat / disengaged

uncertain

sarcastic / skeptical

The text baseline receives effectively identical input.

Compare the text and acoustic results to show whether acoustics actually add information.

5. UI

Extend the existing LiveKit UI.

While the user is talking, show the current acoustic state, for example:

FRUSTRATION 0.72

UNCERTAINTY 0.31

ENERGY 0.64

More importantly, add expression signals to the conversation timeline:

0s 1s 2s 3s 4s 5s

USER



FRUSTRATION



UNCERTAINTY



ENERGY



BACKCHANNEL

▲
mm-hmm

AGENT



We should be able to see:

- user speech,
 - acoustic predictions,
 - backchannel decisions,
 - actual acknowledgement playback,
 - end-of-turn,
 - normal agent response.
-

6. Use Acoustic Information

Use the acoustic signal for at least one real conversational decision.

Examples:

high frustration
→ avoid "go on"

or:

high frustration
→ use fewer acknowledgements

or:

choose a more neutral acknowledgement

or:

change backchannel timing

This is intentionally open-ended.

Explain:

- what behaviour you changed,
- why,

- how you measured whether it helped.

Do not simply add the signal to the LLM prompt and call the problem solved.

7. Benchmark Scenarios

Create repeatable scenarios covering at least:

Same words, different delivery

The acoustic system should distinguish them better than text alone.

Increasing frustration

The user becomes progressively more frustrated.

Hesitant speaker

Pauses, uncertainty, fillers and uneven speaking rate.

High-energy speaker

Fast, energetic delivery.

Flat delivery

Low-energy speech with limited prosodic variation.

Background noise

Repeat a subset under noise.

Long conversation

Expression changes multiple times in one turn.

Hindi or Hinglish

At least one multilingual or code-switched scenario is strongly encouraged.

8. Preserve Conversational Performance

Compare:

Acoustic system OFF

vs:

Acoustic system ON

Measure at minimum:

Metric	OFF	ON	Delta
Response latency P50			
Response latency P95			
EOT latency			
LLM TTFT			
TTS first audio			

Answer:

Does continuously analyzing the user's voice make the normal agent slower?

If yes:

Why?

The acoustic system should not become part of the normal response critical path unless absolutely necessary.

9. Backpressure

Assume audio arrives every:

20 ms

and inference normally keeps up.

Now assume:

normal inference: 40 ms

degraded inference: 300 ms

What happens?

Do not allow an unbounded audio backlog.

Think about:

- bounded queues,
- latest-state processing,
- dropping old windows,
- coalescing work,
- inference cadence,
- memory growth.

A perfect prediction for speech from three seconds ago may no longer be useful.

Demonstrate or test your chosen behaviour.

10. Failure Isolation

Demonstrate what happens when:

Acoustic inference crashes

Does the call continue?

Inference becomes slow

Does the agent wait?

Backchannel generation fails

Does anything else break?

Telemetry becomes unavailable

Does audio processing stop?

GPU/model server becomes unavailable

What does the system degrade to?

We expect graceful degradation.

11. Production System Design

Assume the prototype now needs to support:

10,000 simultaneous voice calls

across multiple enterprise customers.

Each call may involve:

continuous audio

STT

turn detection

acoustic expression inference

backchannel decisioning

LLM

TTS

telemetry

Design the production architecture.

Do **not** implement the entire distributed system.

We want the design and reasoning.

12. Critical vs Non-Critical Path

Clearly identify what directly affects the conversation and what should never block it.

Consider:

STT

turn detection

acoustic inference

backchannel policy

analytics

logs

experiment telemetry

If a non-critical component becomes unhealthy, explain how it is removed from the critical path.

13. Per-Call State

Describe the state maintained per call.

Possible examples:

current speaker

recent audio window

turn state

expression history

backchannel cooldown

pending acknowledgement

LLM context

You decide what actually needs to exist.

Explain:

- where state lives,
 - which state must survive a worker crash,
 - which state is safe to lose,
 - how state is cleaned up when a call ends.
-

14. Model Serving

Assume acoustic inference requires GPU capacity.

Design how you would serve it for thousands of simultaneous streams.

Consider:

- streaming inference,
- batching,
- maximum batching delay,
- model replicas,
- CPU preprocessing,
- GPU utilization,
- autoscaling,
- per-call fairness.

Explain where you choose:

higher throughput

versus:

lower latency

15. Capacity Planning

Estimate requirements for:

1,000 calls

10,000 calls

You do not need perfectly accurate GPU counts.

Start with measurable quantities such as:

audio windows / sec / call

inference duration

batch size

GPU throughput

CPU preprocessing cost

memory per active stream

Show your assumptions.

16. Traffic Spike

Assume normal traffic is:

4,000 calls

and suddenly becomes:

12,000 calls

but available compute cannot handle everything.

Explicitly prioritize:

core conversation

STT / turn detection

acoustic expression

backchanneling

analytics

Explain what gets degraded first and why.

17. Multi-Tenancy

Assume:

Bank A
very conservative backchanneling

Retailer B
frequent acknowledgements

Customer C
acoustic analysis disabled

Customer D
Hindi + English

Customer E
must run entirely on-premise

Explain how you support these differences without creating separate forks.

Also explain how you prevent one large tenant from consuming all inference capacity.

18. Deployment Models

The architecture should support:

Blue Machines Cloud

Customer VPC

Customer on-premise / air-gapped

Explain what changes between environments.

Consider:

- model availability,
 - networking,
 - configuration,
 - observability,
 - upgrades,
 - external APIs,
 - failure recovery.
-

19. Observability

An enterprise customer reports:

“Calls felt slow between 2:00 PM and 2:10 PM.”

Explain how you would determine whether the issue came from:

telephony

audio ingestion

STT

turn detection

acoustic model

backchannel system

LLM

TTS

network

GPU saturation

CPU saturation

Describe the metrics, traces and correlation identifiers you would want.

Do not answer only with:

“Check the logs.”

20. New Model Rollout

You develop a new acoustic model.

It is more accurate but adds:

+35 ms inference latency

Explain how you would roll it out safely.

Consider:

- shadow evaluation,
- percentage rollout,
- tenant-level rollout,
- model versioning,
- latency regressions,
- quality regressions,
- automatic rollback.

Explain what metrics would make you stop the rollout.

21. Architecture Diagram

Provide one architecture diagram showing:

real-time call path

acoustic inference path

normal agent response path

state

model serving

telemetry

failure boundaries

We value clarity over complexity.

22. Tests

Include meaningful automated tests covering:

- streaming audio windows,
 - prediction smoothing,
 - slow inference,
 - failed inference,
 - stale work being dropped,
 - user stops mid-window,
 - rapid speech/silence transitions,
 - expression changes during a turn,
 - acoustic system disabled,
 - normal response unaffected by inference failure,
 - backchannel policy reacting to acoustic state,
 - clean shutdown.
-

23. README Questions

Answer briefly:

1. Why did you choose your acoustic model?
2. What is the earliest point at which you trust its output?

3. What is your true end-to-end detection latency?
 4. How do you know the system isn't mainly learning from words?
 5. How stable are predictions across adjacent windows?
 6. What happens under noise?
 7. What happens across accents and languages?
 8. How do you prevent acoustic inference from slowing the agent?
 9. What happens if inference throughput falls below incoming audio rate?
 10. What would break first at 10,000 concurrent calls?
 11. What would you change for 100,000 calls?
 12. Which capability would you disable first during overload?
 13. Which state needs recovery after a worker crash?
 14. What would you change for an air-gapped deployment?
-

Deliverables

Submit:

1. Working LiveKit implementation
 2. Streaming acoustic-expression component
 3. Integration with the existing backchannel engine
 4. Repeatable benchmark scenarios
 5. Baseline vs acoustic-enabled latency results
 6. Acoustic vs text comparison
 7. UI with expression timeline
 8. Automated tests
 9. Architecture diagram
 10. System-design explanation
 11. README with benchmark findings and trade-offs
-

Evaluation

Real-time systems engineering — 20%

Streaming, async work, cancellation, bounded queues, backpressure.

Acoustic / ML implementation — 15%

Useful real-time signal and sensible model choices.

Benchmark quality — 15%

Measurements are reproducible and conclusions match the evidence.

System design — 20%

Scaling, state, isolation, failure handling, resource management.

UI / observability — 10%

Can we understand what happened visually?

LiveKit integration — 10%

Does it integrate cleanly without damaging the normal voice pipeline?

Testing — 5%

Are difficult real-time cases tested?

Engineering judgment — 5%

Does the candidate understand the limitations and trade-offs?

Design Review

Be prepared for us to change assumptions during the review.

Examples:

GPU capacity is cut by 50%.

One customer suddenly contributes 60% of traffic.

Acoustic inference must move on-premise.

Model inference becomes 5× slower.

We now need 99.99% call availability.

The system needs to support 100,000 concurrent calls.

The acoustic model has twice the accuracy but four times the compute cost.

We care less about an immediate answer and more about how you reason through the consequences.

Core Question

At the end of the project, we should be able to answer:

Can we understand useful conversational signals from the user's voice while they are still speaking, use them to improve the agent, and do so without compromising the latency or reliability of the core voice conversation?

And at production scale:

What breaks first, and how does the system degrade safely when it does?